AQUAPURE WATER PURIFIER COMMERCE PLATFORM

End-to-End System & Deployment Guide

From the architecture and Firebase data model to local development, Vercel APIs, Expo, Android builds, and push notification verification.

Workspace: WaterPurifierProject
Platform: Firebase + Vercel + Expo / React Native
Edition: Project-specific operations manual

Prepared from the current repository configuration and implementation.

How to Use This Guide

This manual explains the application that exists in this workspace, then walks through a clean setup and deployment. It is not a claim that a production environment has already been provisioned. You need your own Firebase project, Vercel project, service-account credentials, and Android build credentials.

> **Security rule:** Never publish a Firebase service-account private key in source code, a mobile build, a browser variable, or a committed environment file. Only the Vercel server functions receive `FIREBASE_SERVICE_ACCOUNT_JSON`.

1. System architecture
2. Repository map
3. Roles and user journeys
4. Firebase data and security
5. Prerequisites and Windows path
6. Provision Firebase
7. Configure environment variables
8. Run the admin app locally
9. Run the Expo mobile app
10. Deploy the admin/API to Vercel
11. Build and install Android
12. Push notification behavior
13. End-to-end verification
14. Troubleshooting
15. Operational limitations

1. System Architecture

AquaPure has three runtime parts. The mobile application and admin website use Firebase's client SDK for sign-in and permitted Firestore operations. The Vercel project hosts the admin website and trusted serverless API routes. Those routes use Firebase Admin SDK credentials on the server for privileged operations and Firebase Cloud Messaging.

Android customer/dealer app

Expo Router + React Native; Firebase Auth and Firestore client; native Android FCM token registration; checkout API calls.

Admin website

React + Vite + TypeScript; Firebase Auth; Firestore listeners and admin-authorized updates; invokes protected Vercel API routes.

Firebase project

Authentication, Cloud Firestore, Security Rules, Android app registration and Firebase Cloud Messaging.

Vercel serverless API

`/api/create-order`, `/api/send-notification`, and `/api/seed-catalog`; Firebase Admin SDK validates identity and performs trusted work.

Mobile app ── Firebase client SDK ──> Firebase Auth / Firestore
Mobile app ── Firebase ID token ──> Vercel API ── Firebase Admin SDK ──> Firestore
Admin website ── Firebase Auth / Firestore ──> admin data
Admin website ── admin ID token ──> Vercel notification API ── FCM ──> Android device

> **Trust boundary:** A Firebase client config (API key, project ID, app ID) identifies the project; it is not an authorization secret. Access is controlled by Firebase Authentication and Firestore Rules. The server service account is privileged and must remain private.

Repository map

Path	Responsibility
`WaterPurifierProject/admin/src/`	Admin Vite application, Firebase client integration, admin order/dealer/service UI.
`WaterPurifierProject/admin/api/`	Vercel serverless APIs for secure checkout, catalogue seeding, and FCM sends.
`WaterPurifierProject/admin/server/firebaseAdmin.ts`	Initializes Firebase Admin using server-only environment variables and verifies admin requests.
`WaterPurifierProject/mobile/app/`	Expo Router routes: authentication, dashboard, shop tabs, profile, and service booking.
`WaterPurifierProject/mobile/services/`	Firebase client, authentication, Firestore shop/order operations, and notifications.
`WaterPurifierProject/mobile/app.json` and `app.config.js`	Expo identity, Android package, plugins, and optional Android Firebase config file detection.
`firestore.rules` at workspace root	Authoritative client database access rules. Publish the complete file to the matching Firebase project.
`WaterPurifierProject/docs/`	Setup, Firebase security, schema, testing, and this end-to-end guide.

2. Roles and User Journeys

Customer

1. Registers with email/password. The app creates a Firebase Auth identity and a `users/{uid}` profile with role `customer` and active account status.
2. Browses active Firestore products, adds items to a local cart, and enters delivery details.
3. Checkout sends product IDs and quantities to Vercel. The server verifies the Firebase ID token, recalculates prices and availability from Firestore, then writes the order transactionally.
4. Reads order/service history from Firestore. An order confirmation is shown locally; later status changes can trigger remote FCM.

Dealer

1. Registers as a dealer with business name and GST number. The user profile and dealer application begin in pending status.
2. An authenticated administrator reviews the application. The admin website updates the application and user profile; approval sets dealer/account status to approved/active.
3. After approval, the mobile app uses dealer pricing and server checkout enforces minimum order quantities.

Administrator

Signs into the web portal with Firebase Authentication. Admin authority is established by a matching `admins/{uid}` record. The admin can seed an empty catalogue, manage products, approve dealer applications, update order/service statuses, and request a protected FCM send.

3. Firebase Data and Security

Collection	Purpose and access pattern
`users/{uid}`	Profile, role, account/dealer state, addresses, and optional native push token. Owner reads their profile and may update allow-listed fields; admins can manage authorized profile data.
`admins/{uid}`	Trusted admin lookup. Provisioned manually through Firebase Console or a secure administrative process; client writes are denied.
`products/{productId}`	Live catalogue, prices and stock. Active products are readable to customers; writes are admin-only. The secure checkout API decrements stock.
`dealerApplications/{appId}`	Dealer application details and review status. Applicant creates/reads their application; admins review it.
`orders/{orderId}`	Customer/dealer orders. Owner and admin reads; client order creation is denied; trusted checkout API creates records; admins may update limited status/payment fields.
`serviceBookings/{bookingId}`	Customer service requests. Active customers create; owner/admin read; admins update the supported status and assignment fields.

Security rules model

- The rules file begins with a default-deny catch-all and opens only explicit collection operations.
- Authentication identifies the caller; the `admins` collection determines administrative access. Never let a client set its own admin role.
- Client-side order creation is explicitly rejected. Checkout uses Firebase Admin on Vercel after validating the ID token, account role/status, cart, products, stock and dealer minimum.
- Firebase Admin SDK bypasses Firestore client Rules. Every server endpoint must therefore validate identity and input in server code; do not treat Rules as a substitute for API authorization.
- Prices use integer paise (100 paise = ₹1) to avoid floating-point rounding.

> **Project alignment matters:** Mobile Firebase config, admin Firebase config, Android `google-services.json`, Vercel `FIREBASE_PROJECT_ID`, and the service-account JSON must all refer to the same Firebase project.

4. Prerequisites and Windows Setup

- Node.js LTS compatible with the repository; Git; VS Code.
- Firebase account/project and Vercel account.
- For Android: a physical Android phone for FCM testing. Android Studio/SDK is optional if you build locally; EAS Build can build remotely.
- For command-line deployment: Vercel account linked to the repository. For native APK distribution with Expo, an Expo account and EAS CLI.

The workspace path contains an ampersand. The project setup notes recommend mapping the repository root to a drive letter to avoid Windows command quoting issues. Run once in PowerShell, selecting an unused drive:

```
subst A: 'C:\Users\ELCOT\Downloads\aquapure---water-purifier-e-commerce-&-admin'
Set-Location A:\WaterPurifierProject
```

If `A:` is already mapped, choose another unused letter. The mapping lasts for the Windows session. Use `npm.cmd` and `npx.cmd` in PowerShell when script execution policy blocks the PowerShell shims.

5. Provision Firebase

1. Create or select one Firebase project. The repository docs currently reference project ID `waterpurifier-9f18a`; if you use another project, replace that ID consistently everywhere.
2. Enable Firebase Authentication and the Email/Password provider.
3. Create a Cloud Firestore database in the required mode/region.
4. Add a Firebase Web App. Copy its config values into the admin and mobile environment variables described below.
5. Add an Android app with package name `com.waterpurifier.ecommerce`. Download its `google-services.json` into `WaterPurifierProject/mobile/`. The current dynamic Expo config detects this file and adds it to Android config.
6. Enable Firebase Cloud Messaging for the project and make sure the backend service identity is permitted to send messages.
7. Publish the entire root `firestore.rules` in Firebase Console > Firestore Database > Rules. Do not replace it with a permissive test rule.

Provision the first administrator

1. Create the administrator's Firebase Auth account through the app's registration flow or Firebase Authentication.
2. Copy that account's UID from Firebase Authentication.
3. In Firestore Console, set the corresponding `users/{uid}` profile role to `admin` as a trusted console operation.
4. Create `admins/{uid}` with administrative metadata. The application does not permit client-side creation or modification of admin records.

6. Configure Environment Variables

Admin local environment

Create `WaterPurifierProject/admin/.env.local` with the Firebase Web App settings:

```
VITE_FIREBASE_API_KEY=your_web_api_key
VITE_FIREBASE_AUTH_DOMAIN=your_project_id.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your_project_id
VITE_FIREBASE_STORAGE_BUCKET=your_project_id.firebasestorage.app
VITE_FIREBASE_MESSAGING_SENDER_ID=your_sender_id
VITE_FIREBASE_APP_ID=your_web_app_id
```

Mobile local environment

Create `WaterPurifierProject/mobile/.env` using the same Firebase Web App values and the deployed API origin:

```
EXPO_PUBLIC_FIREBASE_API_KEY=your_web_api_key
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=your_project_id.firebaseapp.com
EXPO_PUBLIC_FIREBASE_PROJECT_ID=your_project_id
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=your_project_id.firebasestorage.app
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=your_sender_id
EXPO_PUBLIC_FIREBASE_APP_ID=your_web_app_id
EXPO_PUBLIC_API_BASE_URL=https://your-vercel-domain.vercel.app
```

Do not append a slash to `EXPO_PUBLIC_API_BASE_URL`. Values prefixed with `EXPO_PUBLIC_` are bundled into the mobile client and must not contain secrets.

Vercel server environment

Set these in Vercel Project Settings > Environment Variables for the environments you deploy:

```
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=...
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_STORAGE_BUCKET=...
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
FIREBASE_PROJECT_ID=your_project_id
FIREBASE_SERVICE_ACCOUNT_JSON={...service account JSON...}
```

> **Do not add `FIREBASE_SERVICE_ACCOUNT_JSON` to an admin `VITE_*` variable or mobile `EXPO_PUBLIC_*` variable.** It is a server secret only. Restrict who can view it in Vercel and rotate it if exposed.

7. Run the Admin Website Locally

```
Set-Location A:\WaterPurifierProject\admin
npm.cmd install
npm.cmd run dev
```

Open the local URL printed by Vite (the project script uses port `3000`). The admin interface uses Firebase client sign-in and Firestore. The Vite development server does **not** execute Vercel's `admin/api/` functions. For checkout/catalogue/notification APIs, use the deployed Vercel URL in mobile config, or run Vercel's local development server from the admin project after installing the Vercel CLI and configuring server credentials.

Before deploying, validate the admin app:

```
npm.cmd run build
```

The build runs TypeScript checking and Vite production bundling.

8. Run the Expo Mobile App

```
Set-Location A:\WaterPurifierProject\mobile
npm.cmd install
npx.cmd expo start
```

Scan the terminal QR code with Expo Go on a device on the same network, or press `a` if an Android emulator is configured. If port `8081` is already in use, accept Expo's prompt to use another port. Expo Go is useful for UI and basic app flows.

Check SDK dependency compatibility before native builds:

```
npx.cmd expo install --check
```

Resolve any reported package-version mismatches with Expo's installer before considering a production Android build. Local `.env` changes require restarting the Expo bundler.

9. Deploy the Admin Website and API to Vercel

1. Push the repository to a Git provider accessible to Vercel.
2. In Vercel, import the repository. Set the project Root Directory to `WaterPurifierProject/admin`.
3. Confirm the Vercel build uses `npm run build` and output directory `dist`. The checked-in `vercel.json` configures the Vite build and SPA rewrite; `api/` files become serverless functions.

4. Add all six `VITE_FIREBASE_*` variables plus `FIREBASE_PROJECT_ID` and `FIREBASE_SERVICE_ACCOUNT_JSON`. Make sure the service account's project matches the client config and has the Firestore and FCM permissions required by the API.
5. Deploy. Open the Vercel deployment URL and verify the admin sign-in page loads.
6. Set mobile `EXPO_PUBLIC_API_BASE_URL` to the Vercel origin (no trailing slash). For cloud builds, also set public Firebase/API values in the EAS environment used by the build.

Available server routes in the current project:

Route	Purpose	Caller
`POST /api/create-order`	Validates Firebase identity and checkout payload; prices and stock are re-read server-side; order/address/stock updates are transactional and idempotent.	Signed-in mobile customer/dealer.
`POST /api/send-notification`	Looks up the recipient's saved device token and sends an FCM message after verifying the caller is an admin.	Admin website only.
`POST /api/seed-catalog`	Admin-authenticated sample catalogue import.	Admin website only; use when appropriate for an empty catalogue.

10. Build and Install Android

The app's Android application ID is `com.waterpurifier.ecommerce`. Native push support requires an Android development/production build configured with the matching Firebase Android app and `google-services.json`; Expo Go does not provide this app's native FCM registration path.

Build an internal APK with EAS

1. Install/login to EAS from the mobile project:

```
Set-Location A:\WaterPurifierProject\mobile
npx.cmd eas-cli@latest login
npx.cmd eas-cli@latest build:configure
```

2. Use or create an internal APK profile in `eas.json`:

```
{
  "build": {
    "preview": {
      "distribution": "internal",
      "android": { "buildType": "apk" }
    }
  }
}
```

3. Ensure the cloud build receives the required environment values and Android Firebase config. The repo ignores local `.env*` and `google-services.json`; ignored files are not automatically available to EAS. Set public values in EAS. Include the Android config file in the private build input using a controlled `.easignore` strategy or EAS file variable; never commit secrets.
4. Build and install the generated APK:

```
npx.cmd eas-cli@latest build --platform android --profile preview
```

Download the APK from the EAS build page, transfer it to the test phone, allow installation from that source, and install. Sign in and accept Android's notification permission prompt. Direct APK installation is different from publishing to Google Play.

11. Push Notification Behavior

1. On a physical Android device in a native build, the mobile app requests notification permission, creates the `orders` channel, gets the native FCM device token, and writes `pushToken` and `pushTokenUpdatedAt` to the signed-in user's own profile.
2. Order placement and service booking currently show a local confirmation notification from the app.
3. When an admin updates an order, service booking, or dealer decision, the admin UI saves the Firestore change and calls `/api/send-notification`.
4. The API verifies the Firebase ID token and requires an `admins/{uid}` record, reads the recipient token, and sends via Firebase Admin Messaging.

> **Important current behavior:** The checkout endpoint does not send a remote push for a newly created order. A successful local order confirmation is separate from an admin-driven remote status notification. Failed sends are returned as errors; there is no retry queue, so a message missed because no token exists is not delivered later automatically.

12. End-to-End Verification Checklist

- Admin and mobile clients show the same Firebase project ID; Vercel service account and Android app are in that project.
- Firestore Rules are published from the full root `firestore.rules`; no test-mode allow-all rules remain.
- Register a customer and verify a `users/{uid}` profile with active customer role/status.
- Sign in on the installed Android build; grant notifications and confirm a non-empty `pushToken` in the user's Firestore document.
- As an admin, import/create active products and verify mobile catalogue shows Firestore products.
- Place a test COD order. Confirm one order document is created, prices are server-derived, stock decrements once, and a retry does not duplicate the order.
- Change that order's status in admin. Confirm status/timeline updates and the device receives an FCM notification.
- Submit a dealer application, register the dealer token in a native Android build, then approve it from admin and verify the status update and notification.
- Book a service request, update its status in admin, and verify the corresponding remote message.
- Review Vercel Function logs and Firebase Authentication/Firestore logs if an API call fails.

13. Troubleshooting

Symptom	Likely cause and next check
No push token appears	Expo Go, emulator, denied notification permission, no native Android config, or sign-in/profile failure. Use a physical device and native build; reopen the app and verify Firebase project/package match.
“No registered device token”	The remote push could not be sent. Register the user on an installed Android build before triggering the admin action. A previous failed send is not queued.
FCM project mismatch	Android `google-services.json` and Vercel service account/client Firebase config refer to different Firebase project IDs. Reconfigure and reinstall so the device gets a token from the right project.
Admin API returns 401/403	Admin ID token missing/expired or the caller UID has no `admins/{uid}` record. Confirm the admin is signed into the expected Firebase project.

Symptom	Likely cause and next check
Checkout API not configured	Mobile `EXPO_PUBLIC_API_BASE_URL` is missing, stale, or includes an incorrect origin. Restart/rebuild the app after changing build-time public values.
Checkout returns server error	Check Vercel environment variables, service-account JSON validity, Firestore IAM permissions, product data, and function logs.
Admin page loads but API is 404 locally	Vite alone serves the frontend, not Vercel functions. Use the deployed API URL or run the Vercel dev server.
Expo asks to use another port	The default Metro port is occupied. Accept another port and scan the new QR code from the same local network.

14. Cost, Release, and Operational Limits

- Firebase products have plan-specific quotas and billing rules. Review the current Firebase plan and usage before production traffic.
- FCM message delivery does not have a per-message charge under standard Firebase Cloud Messaging use, but associated Firebase services and infrastructure remain subject to their plan/terms.
- Vercel's free plan and EAS free build allowance have current limits and terms. Confirm eligibility, especially for commercial use, before relying on them for a business service.
- Installing an APK directly can avoid Google Play distribution, but users must install updates manually. Google Play publishing requires a developer account and compliance with Play policies.
- The checkout currently records Cash on Delivery and dealer-credit orders; it does not charge cards or integrate a payment provider.
- Only physical/native Android testing proves the complete FCM path. TypeScript/build success alone cannot confirm Firebase credentials, OS permission state, or real device delivery.

15. Recommended Deployment Order

1. Create/configure Firebase and publish the complete Firestore Rules.
2. Bootstrap the first admin account and provision its `admins/{uid}` document.
3. Configure Vercel server secrets and deploy the admin website/API.
4. Configure local mobile Firebase/API values and verify sign-in against the deployed backend.
5. Add Android Firebase configuration, build/install the native APK, and register a token.
6. Seed the catalogue, place a test order, update status, and verify FCM delivery.
7. Only after the complete test passes, plan release, monitoring, backup, support, and billing controls.